1 Problem

Given a set of M deterministic black-box functions $\{f_1, f_2, ..., f_m\}$, where each of these are approximations to a target objective function, g, and a high-dimensional design space $x \in \mathbb{S}^{n-1}$ (a hypersphere in $\mathbb{R}^d$), develop methods to select a function f_j and design point x to maximize the expected value of the true function $g(x)$ under a cost-constrained evaluation budget. Each function f_j has an associated evaluation cost c_j and known epistemic uncertainty σ_j, quantifying its reliability at design point x.

There exist these constraints that must be considered:

- The true function $g(x)$ is unknown and can never be discovered, only estimated with a surrogate function that we can evaluate as a function, $f_j(x)$, where $1 \leq j \leq m$.
- Function evaluations are deterministic, but epistemic uncertainty varies across both j. (f_j's are equal to g with an error sampled from a gaussian distribution with variance as a property of j, and seeded by x. i.e. $f_j(x) = g(x) + \epsilon_j(x)$, where $\epsilon_j(x) = GaussianSample(\mu = 0, \sigma = \sigma_j, seed = hash(x))$
- There is a limited and fixed budget that allows for a limited number of functions, f_j, evaluations
- The input space is restrained to some $n - dim$ hypersphere

2 Approaches

Single-Shot

Our goal is to maximize the expected value of a target function g, but since we only have access to approximations f_j, we instead optimize over these while incorporating a penalty for uncertainty. Specifically, we now define our acquisition function as:

$$f_j(x) - \lambda \cdot \sigma_j(x)$$

For simplicity, we set $\lambda = 1$, although tuning this hyperparameter could be a valuable future direction. This setup reflects the classic exploration-exploitation tradeoff.

In a single-shot budgeted scenario, we restrict our choices to those where the cost c_j is within a fixed budget B, and the input x lies on the n-dimensional unit hypersphere, $\mathbb{S}^{n-1}$. Thus, our optimization problem becomes:

$$(j^*, x^*) = \arg\max_{j,x} [f_j(x) - \sigma_j(x)] \quad \text{subject to} \quad c_j \leq B, \quad x \in \mathbb{S}^{n-1}$$

Here, $f_j(x) - \sigma_j(x)$ serves as our acquisition function that balances predicted value and epistemic uncertainty.

Multi-Shot

We wish to maximize the true objective g by sequentially querying approximations f_j under a fixed total cost budget B. Each f_j has evaluation cost c_j and epistemic uncertainty $\sigma_j(x)$. We use the simple acquisition function

$$a(j, x) \;=\; f_j(x) \;-\; \sigma_j(x),$$

with $\lambda = 1$, to trade off predicted value and uncertainty. Let remaining budget $R_1 = B$ and history

$$\mathcal{D}_t = \{(j_\tau, x_\tau, y_\tau)\}_{\tau=1}^{t-1}, \quad y_\tau = f_{j_\tau}(x_\tau).$$

At each iteration t we

$$(j_t, x_t) = \arg \max_{\substack{1 \le j \le m,\, x \in \mathbb{S}^{n-1} \\ c_j \le R_t}} a(j,x), \quad y_t = f_{j_t}(x_t), \quad R_{t+1} = R_t - c_{j_t},$$

append (j_t, x_t, y_t) to $\mathcal{D}_{t+1}$, and stop when $R_{t+1} \le 0$. The best found is

$$(j^*, x^*) = \arg \max_{(j_\tau, x_\tau) \in \mathcal{D}_{T+1}} f_{j_\tau}(x_\tau).$$

2.0.1 Gaussian Process (GP) Method

We fit a Gaussian process to each f_j using data in $\mathcal{D}_t$, yielding posterior mean $\mu_{j,t}(x)$ and variance $s_{j,t}^2(x)$. We then redefine

$$a(j,x) = \mu_{j,t}(x) \; - \; s_{j,t}(x),$$

so that points with high predicted value and low variance are preferred. We select (j_t, x_t) by maximizing $a(j,x)$ subject to $c_j \le R_t$. After evaluation we update only the GP for index j_t.

2.0.2 Multi-Armed Bandit (MAB) via Thompson Sampling

We implement a shared Bayesian model over all (j,x), for example a linear model on features $\phi(j,x)$. So at each t we draw a sample $\tilde{g}_t$ from the posterior, compute

$$a(j,x) = \tilde{g}_t(x) \; - \; \sigma_j(x),$$

and pick the feasible maximizer under $c_j \le R_t$. Observation y_t then updates the posterior. Random draws of $\tilde{g}_t$ naturally balance exploration and exploitation as observed with extensive discovery in the literature.

2.0.3 Greedy Algorithm to Select Functions

At a general level, finding the optimal sequence $\{(j_t, x_t)\}_{t=1}^T$ under $\sum_t c_{j_t} \le B$ is a combinatorial optimization problem over all possible function-point pairs. This quickly becomes intractable when m is large or x is continuous. To avoid this, we use a simple greedy heuristic:

1. Initialize remaining budget $R_1 = B$ and data set $\mathcal{D}_1 = \emptyset$.
2. For $t = 1, 2, \ldots$ while $R_t > 0$:
   - Compute the acquisition
   $$a(j,x) = f_j(x) - \sigma_j(x) \quad \text{for all } 1 \le j \le m,\; x \in \mathbb{S}^{n-1} \text{ with } c_j \le R_t.$$
   - Select
   $$(j_t, x_t) = \arg \max_{c_j \le R_t} a(j,x).$$

- Evaluate $y_t = f_{j_t}(x_t)$, append (j_t, x_t, y_t) to $\mathcal{D}_{t+1}$.
- Update $R_{t+1} = R_t - c_{j_t}$.

3. End loop when $R_{t+1} \le 0$.

This greedy policy respects the cost budget, focuses each evaluation on the currently most promising pair, and naturally balances exploration and exploitation via the $-\sigma_j(x)$ term.

2.0.4 Combinatorial Optimization

Decision variables

$$z_{j,i} \;=\; \begin{cases} 1, & \text{if we select approximation } f_j \text{ at design } x^{(i)}, \\ 0, & \text{otherwise,} \end{cases}$$

for $j = 1, \ldots, m$ and $i = 1, \ldots, N$.

Objective Maximize total acquisition value:

$$\max_{z} \sum_{j=1}^{m} \sum_{i=1}^{N} \left[f_j\left(x^{(i)}\right) - \lambda \sigma_j\left(x^{(i)}\right) \right] z_{j,i} \qquad (\lambda = 1)$$

Constraints

$$\sum_{j=1}^{m} \sum_{i=1}^{N} c_j z_{j,i} \le B \qquad \text{(budget constraint)}$$

$$\sum_{j=1}^{m} z_{j,i} \le 1, \quad i = 1, \ldots, N \qquad \text{(at most one evaluation per point)}$$

$$z_{j,i} \in \{0, 1\}, \quad j = 1, \ldots, m, \; i = 1, \ldots, N \qquad \text{(binary variables)}$$

3 Simulated Results

Set Up

To evaluate the approaches, I've designed a synthetic optimization problem based on the framework. The experimental setup consists of a target function (our g), which is a sum of many Gaussian peaks creating a surface along the unit hypersphere we're considering, approximations (our f_j functions), and we introduce these functions with a varying amount of values.

- Cost: Linearly spaced from 1 to 3 units across the m functions
- Noise: linearly spaced from 0.2 to 1
- $f_j(x) = g(x) + \epsilon_j(x)$, where $\epsilon_j(x)$ is seeded by hash(x)
- $\sigma_j(x) = noise \cdot (1 + 0.5|$

Implementation Details

1. dimensions $\in$ 128
2. number of approximations $\in$ 100
3. 50 independent trials per config
4. 1000 design points (values for x)
5. budget $\in$ 100

All methods and code is in python with these key components. Hypersphere sampling for designs. The methods are:

- Greedy Sequential: Pre-computes acquisition values and uses cost-adjusted acquisitions
- Multi-Shot GP: Maintains a separate GP for each approximation.
- Thompson Sampling: Samples from the Gp posterior, then uses that to supply the acquisition function ($\tilde{g}(x) = \mu_j(x) + \lambda s_j(x) \times \mathcal{N}(0, 1)$

4 Conclusion

This presents my approaches to solving this problem in a cost-constrained optimization problem using multiple approximation functions with known epistemic uncertainties.

The natural multi-fidelity Bayesian optimization approach is the natural formulation and reasoning behind a lot of this formulation. It helped develop the trade-offs in what I believe the simulation favored for cheaper and noisier approximations with costly evaluations.

Future Directions. Looking ahead, I'd be interested in exploring dynamic λ tuning and more computationally efficient surrogate models. Overall, this problem served to introduce various probabilistic sampling techniques and the path to solving them.

5 Sources

[1] arxiv.org/pdf/2211.02732

[2] scipedia.com/.../Draft_Sanchez_Pinedo_821673761pap_1820.pdf

[3] NeurIPS 2020: f05da679342107f92111ad9d65959cd3

[4] NeurIPS 2012: 05311655a15b75fab86956663e1819cd

[5] people.maths.bris.ac.uk/~maajg/teaching/stochopt/thompson.pdf

[6] arxiv.org/pdf/2007.06823